
2-Mark Questions – Half Page Answers

---

1. Who developed React?

React was developed by **Jordan Walke**, a software engineer at **Facebook**, in **2011**. Initially, it was used in **Facebook's News Feed** and later on **Instagram** to enhance user experience through faster UI updates. React introduced the concept of the **Virtual DOM**, which improved performance in large-scale applications. In **2013**, React was released as **open-source**, allowing developers worldwide to contribute. Today, React is maintained by **Meta** and a large **community**, making it one of the most popular tools for building **single-page applications (SPAs)** and **reusable UI components**.

---

2. What is the full form of JSX?

JSX stands for **JavaScript XML**. It is a **syntax extension** for JavaScript that allows developers to write **HTML-like elements** directly inside JavaScript code. JSX helps combine the **markup** and **logic** of components, making them **readable** and **maintainable**. Although it looks like HTML, JSX is compiled by **Babel** into React.createElement() calls, which create **Virtual DOM elements**. JSX also supports embedding **JavaScript expressions** inside {} for dynamic rendering.
Example:

```
const name = 'Arya';

const element = <h1>Hello, {name}</h1>;
```

---

3. Which command is used to create a React app?

The recommended command is:

```
npx create-react-app myapp
```

Here:

- **npx** → Runs a package without installing globally.
- **create-react-app** → Sets up a complete React environment with Webpack, Babel, and a development server.
  Once created, navigate to the project folder:

```
cd myapp

npm start
```

This starts a local server (default: **localhost:3000**) where the app runs. This method ensures using the latest version without manual setup.

---

4. Is React a library or a framework?

React is a **JavaScript library** for building **user interfaces**. It focuses only on the **View** layer of the **MVC architecture**. React promotes a **component-based architecture**, enabling developers to build **reusable UI components**. It does not include features like routing or state management by default, but it can be extended using libraries such as **React Router** or **Redux**. Unlike frameworks like Angular, React is **flexible** and can be integrated with other tools and technologies depending on project needs.

---

5. What is the Virtual DOM?

The **Virtual DOM (VDOM)** is an **in-memory copy** of the real DOM. Instead of updating the browser DOM directly, React first updates the Virtual DOM and compares it to the previous version (**diffing**). Only the changed elements are updated in the real DOM (**patching**). This improves **performance** by reducing costly DOM manipulations.
Example: Changing <h1>Hello</h1> to <h1>Hello, World!</h1> updates only the text content, not the entire element.

---

6. What is the extension of a React component file?

React component files usually have:

- **.js** → JavaScript components
- **.jsx** → JavaScript with JSX syntax
- **.tsx** → TypeScript with JSX syntax
  Using .jsx or .tsx helps editors and compilers provide better **syntax highlighting**, **type checking**, and **error detection** for React-specific code.

---

7. Name one hook in React.

An example is **useState**. It allows adding **state** to functional components. It returns an array with the current state value and a function to update it.
Example:

const [count, setCount] = useState(0);

Here, count is the state variable, and setCount is used to update it.

---

8. Purpose of React Developer Tools

React Developer Tools is a **browser extension** for debugging React applications. It allows developers to:

- Inspect **component hierarchy**
- View and edit **props** and **state**

- Monitor component re-renders
  It is available for Chrome and Firefox and is essential for troubleshooting and performance optimization.

---

9. What does CRA stand for?

CRA stands for **Create React App**. It is an official tool for quickly setting up a React project with all necessary configurations like **Webpack**, **Babel**, and **ESLint**. CRA simplifies development by including scripts for **starting**, **building**, and **testing** applications.

---

10. Write a simple JSX syntax.

Example:

const element = <h1>Hello, World!</h1>;

This is a JSX expression that looks like HTML but is actually JavaScript. It will be compiled into React.createElement() calls and rendered to the **Virtual DOM**.

---

11. What is the latest version of React (2025)?

As of 2025, the latest stable version is **React 19**. This version introduces improved **Server Components**, better **performance optimizations**, and enhanced **React Suspense** features for asynchronous rendering.

---

12. What function is used to render elements in React?

React uses the **ReactDOM.render()** function to render elements into the DOM.
Example:

ReactDOM.render(<App />, document.getElementById('root'));

Here, <App /> is rendered into the HTML element with the ID root.

---

13. What keyword is used to define a class component?

In React, a **class component** is defined using the **class** keyword in JavaScript, followed by extending the **React.Component** base class. A class component must have a **render()** method that returns JSX.
Example:

```
class MyComponent extends React.Component {
  render() {
    return <h1>Hello World</h1>;
  }
```

}

Class components can have **state** and **lifecycle methods**, unlike older functional components without hooks.

---

14. Name a lifecycle method in class components.

One common lifecycle method is **componentDidMount()**.

- It executes **after** the component is mounted (inserted into the DOM).
- Commonly used for **API calls**, subscriptions, or initial setup.
  Example:

```
componentDidMount() {
  console.log("Component loaded");
}
```

---

15. Can we return multiple elements from a component?

Yes, in React we can return multiple elements using **Fragments** (<></>) or an array. Fragments prevent adding extra nodes to the DOM.
Example:

```
<>
  <p>Line 1</p>
  <p>Line 2</p>
</>
```

This allows grouping without unnecessary wrapper divs.

---

16. What is Babel in context of React?

Babel is a **JavaScript compiler** used in React projects. It converts modern JavaScript (ES6+) and JSX into code compatible with older browsers.

- JSX → React.createElement()
- ES6+ syntax → ES5 syntax
  This ensures React code works in all environments.

---

17. Write one difference between HTML and JSX.

In HTML, we use the **class** attribute for CSS, but in JSX, we use **className**.
Example:
HTML:

JSX:
This change avoids conflicts with JavaScript's class keyword.

---

18. Can props be modified inside a child component?

No, **props** are **read-only**. They are passed from a **parent** component to a **child** and cannot be modified inside the child. If modification is needed, state should be used or a callback passed to the parent.

---

19. What is the default port number for a React app?

By default, Create React App runs the development server on **port 3000**. The app is accessed at:

http://localhost:3000

The port can be changed by setting the PORT environment variable.

---

20. What is React?

React is a **front-end JavaScript library** for building **user interfaces**. It uses a **component-based architecture** and the **Virtual DOM** for fast updates. React is ideal for **Single Page Applications (SPAs)** and supports building cross-platform mobile apps using **React Native**.

---

21. Define Virtual DOM and its use.

The **Virtual DOM** is a lightweight, in-memory representation of the real DOM. React updates the Virtual DOM first, compares it with the old version (**diffing**), and then updates only the changed parts in the real DOM (**patching**). This process improves **performance** by reducing unnecessary DOM operations.

---

22. What are the key features of React?

- **Component-Based Architecture** – reusable, independent UI components
- **Virtual DOM** – efficient updates
- **JSX** – HTML-like syntax in JS
- **Unidirectional Data Flow** – predictable data handling
- **Strong Community Support**

---

23. Explain JSX with an example.

JSX allows writing HTML-like syntax in JavaScript.
Example:

```
const name = "Arya";

const element = <h1>Hello, {name}</h1>;
```

This combines markup and logic, making code cleaner. Babel compiles JSX into React.createElement() calls.

---

24. List differences between HTML and JSX.

HTML	JSX
Uses class	Uses className
Attribute names are lowercase	CamelCase for attributes (e.g., onClick)
Tags can be unclosed	Tags must be closed

---

25. What is the purpose of props in React?

Props allow passing **data** from parent to child components. They make components **reusable** and **configurable**. Props are **immutable**, meaning they cannot be changed by the receiving component.

---

26. Differentiate between Functional and Class components.

Functional	Class
Simple JS functions	ES6 classes
Use hooks for state	Use this.state and lifecycle methods
Easier to write	More verbose

---

27. Explain React Developer Tools briefly.

React Developer Tools is a **browser extension** for Chrome and Firefox. It allows inspecting **React components**, their **props**, and **state**, and helps debug performance issues.

---

28. How do you pass data using props?

Pass data as attributes in JSX:

```
<Welcome name="Arya" />
```

Inside the child:

```
function Welcome(props) {
  return <h1>Hello {props.name}</h1>;
```

}

---

29. What is the use of ReactDOM?

ReactDOM connects React components to the browser DOM.
Example:

ReactDOM.render(<App />, document.getElementById('root'));

---

30. What is Create React App (CRA)?

CRA is an official CLI tool for quickly setting up a React project with all necessary configurations (Webpack, Babel, ESLint). It comes with commands for starting, building, and testing apps.

---

31. Why is JSX used in React?

JSX makes code **readable**, allows embedding **JavaScript expressions**, and helps combine UI and logic in one file.

---

32. Write syntax of a simple functional component.

function Greet() {

  return <h1>Hello World</h1>;

}

---

33. What is the importance of keys in lists?

Keys help React identify which list items have changed, improving rendering performance.

---

34. Explain useState with an example.

const [count, setCount] = useState(0);

This hook stores state in a functional component. Clicking a button can update the value.

---

35. What are the rules for writing JSX?

- Return one parent element
- Use className instead of class
- All tags must be closed
- Use {} for JS expressions

---

36. Difference between state and props.

Props	State
Read-only	Mutable
Passed from parent	Internal to component

---

37. How does rendering work in React?

React renders components into the **Virtual DOM**, compares with previous renders (**diffing**), and updates only the changed parts in the real DOM.

---

38. What happens if you modify props?

Props are **immutable**. Attempting to modify them directly will cause unexpected behavior or errors.

---

39. Explain React app folder structure.

- **public/** → static files (index.html)
- **src/** → components, App.js
- **node_modules/** → dependencies
- **package.json** → project config

---

40. What are fragments in React?

Fragments let you return multiple elements without extra wrapper tags:

```
<>
  <p>Item 1</p>
  <p>Item 2</p>
</>
```

4-Mark Questions – Full Page Answers

---

1. Explain JSX in detail with an example.

JSX stands for **JavaScript XML**. It is a **syntax extension** for JavaScript used in React to describe the UI. With JSX, developers can write **HTML-like code** directly inside JavaScript files. JSX improves **readability**, as the structure and logic of components remain in one place. Though it looks like HTML, JSX is **not valid JavaScript** by itself — it must be **compiled** (by tools like **Babel**) into React.createElement() calls, which produce **Virtual DOM elements**.

JSX allows embedding **JavaScript expressions** inside curly braces {} for dynamic rendering. For example:

const name = "Arya";

const element = <h1>Hello, {name}</h1>;

Here, {name} will be replaced by the value of the name variable. JSX follows certain rules:

1. Must return **a single parent element**.
2. Use **className** instead of class.
3. All tags must be **properly closed**.

Conclusion: JSX is not mandatory in React, but it simplifies UI creation and makes code easier to understand and maintain.

---

2. Difference between HTML and JSX.

HTML and JSX may look similar, but they have important differences:

1. **Syntax** – HTML is a **markup language**, whereas JSX is a **JavaScript syntax extension** that produces React elements.
2. **Attributes** – In HTML, attribute names are **lowercase** (onclick), while in JSX they follow **camelCase** (onClick).
3. **Class vs className** – HTML uses class for CSS, JSX uses **className** to avoid conflicts with the class keyword in JavaScript.
4. **Tag closure** – HTML tags may remain unclosed (<img>), but in JSX all tags must be closed (<img />).
5. **Expressions** – HTML does not allow JavaScript expressions directly, JSX allows embedding them within {}.

Example:

<!-- HTML -->

// JSX

Conclusion: JSX brings the power of JavaScript into HTML-like syntax, allowing dynamic, component-based UI creation.

---

3. Purpose of Props in React.

Props (short for **Properties**) are **read-only attributes** used to pass data from a **parent component** to a **child component**. They help make components **reusable** and **configurable** without changing their internal logic.

For example:

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
<Welcome name="Arya" />
```

Here, name="Arya" is passed as a prop. Props are **immutable** — they cannot be changed by the child component.

Advantages of Props:

1. Promote **reusability** of components.
2. Ensure **unidirectional data flow** (from parent to child).
3. Allow easy configuration of components.

Conclusion: Props are essential in React for passing data and customizing components without altering their behavior.

---

4. Differentiate between Functional and Class Components.

In React, **functional components** are JavaScript functions that return JSX, while **class components** are ES6 classes that extend React.Component and use a render() method.

Functional Components:

- **Syntax:** Simple functions returning JSX.
- **State Management:** Use **hooks** like useState and useEffect.
- **Performance:** Generally faster and easier to test.
  Example:

```
function Greet() {
  return <h1>Hello</h1>;
}
```

Class Components:

- **Syntax:** Use the class keyword and render() method.

- **State Management:** Use this.state and lifecycle methods like componentDidMount. Example:

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello</h1>;
  }
}
```

Key Differences:

1. **Syntax:** Functions vs. classes.
2. **State:** Hooks vs. this.state.
3. **Lifecycle:** Hooks replace lifecycle methods in modern React.

Conclusion: Functional components are preferred in modern React due to their simplicity and hooks support.

---

5. Explain React Developer Tools in detail.

React Developer Tools is a **browser extension** (for Chrome and Firefox) used for **debugging React applications**. It allows developers to:

1. Inspect the **component tree**.
2. View and edit **props** and **state** in real time.
3. Monitor **component re-renders** to detect performance issues.

Features:

- **Components Tab:** Shows the hierarchy of components and their props/state.
- **Profiler Tab:** Measures rendering time and performance impact of components.

Benefits:

- Helps in identifying unnecessary re-renders.
- Debugging becomes easier as you can simulate prop/state changes.

Conclusion: React Developer Tools is essential for efficient debugging and performance optimization during development.

---

6. Explain the working of Virtual DOM with an example.

The **Virtual DOM** is a lightweight, in-memory copy of the real DOM. When a component's state or props change, React updates the **Virtual DOM** instead of directly updating the real DOM.

Process:

1. **Render Phase:** React updates the Virtual DOM.
2. **Diffing:** The updated Virtual DOM is compared to the previous one to detect changes.
3. **Patching:** Only the changed elements are updated in the real DOM.

Example: Changing <h1>Hello</h1> to <h1>Hello World</h1> updates only the text, not the entire element.

Advantages:

- Faster updates.
- Reduced browser reflows.

Conclusion: Virtual DOM ensures React apps are fast and efficient, even with frequent updates.

---

7. Explain the folder structure of a React app.

A default **Create React App** project has:

1. **node_modules/** – Stores installed dependencies.
2. **public/** – Contains index.html and static assets like images.
3. **src/** – Main source folder containing:
    - App.js → Root component.
    - index.js → Entry point; renders App to the DOM.
    - Additional components.
4. **package.json** – Lists dependencies and scripts.
5. **package-lock.json** – Dependency version lock file.
6. **.gitignore** – Files to ignore in Git.

Conclusion: The folder structure keeps React apps organized and maintainable.

---

8. Explain useState Hook with an example.

useState is a React Hook used in **functional components** to manage state. It returns an array containing the **current state** and a **function to update it**.

Syntax:

```
const [count, setCount] = useState(0);
```

Example:

```
function Counter() {
  const [count, setCount] = useState(0);
  return (
```

```
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

Here, clicking the button updates the state and re-renders the component.

Conclusion: useState makes functional components powerful by adding state-handling capability.

---

9. Explain State vs Props in detail.

Feature	State	Props
Mutability	Mutable	Immutable
Ownership	Owned by component	Passed from parent
Update	Via setState / Hooks	Cannot be changed by child
Purpose	Store local data	Pass data/configuration

Example:

- **State:** A counter value that changes when a button is clicked.
- **Props:** Button label passed from parent to child.

Conclusion: State manages internal data, while props allow data transfer between components.